Pathfinding on Occupancy Grids

Part 1 — Motivation: Why Pathfinding?

Imagine building a mobile robot navigating a warehouse with shelves and walls. It has a map, but how does it choose a route? This is the **pathfinding problem**: given a map with obstacles, find a collision-free (ideally shortest) route from start to goal. It's used everywhere — warehouse robots, game AI, self-driving cars, evacuation planning.

Part 2 — Representing the World: Occupancy Grids

2.1 What Is an Occupancy Grid?

A 2D matrix where every cell is either **1 (free/passable)** or **0 (occupied/obstacle)**. Think of it as graph paper over a floor plan.

2.2 Converting an Image to a Grid

Real-world maps come as images. The conversion pipeline:

```
Grayscale Image → Thresholding → Binary Grid (0s and 1s)
```

Step 1 — Load as grayscale: Each pixel is 0 (black) to 255 (white).
Step 2 — Threshold: Pixels above a cutoff → 255 (free), below → 0 (blocked).
Step 3 — Normalize: Divide by 255 so values are 0 or 1.

2.3 The Code

```
import cv2
import numpy as np

def create_occupancy_grid_from_image(image_path, threshold=128):
    # Load as grayscale: each pixel is 0-255
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    # Threshold: pixels > 128 become 255 (free), rest become 0 (blocked)
    _, thresholded = cv2.threshold(image, threshold, 255, cv2.THRESH_BINARY)

    # Normalize: 255→1, 0→0
    occupancy_grid = thresholded // 255

    return occupancy_grid, image
```

Key functions:

- `cv2.imread(path, cv2.IMREAD_GRAYSCALE)` — loads image as a NumPy 2D array
- `cv2.threshold(image, thresh, maxval, type)` — binary thresholding

Threshold tuning: Lower threshold = more free space (permissive). Higher = less free space (conservative). 128 is the midpoint default.

Part 3 — Graph Search Fundamentals

3.1 The Grid IS a Graph

Every free cell is a **node**. Each node connects to its 4 cardinal neighbors (if they're also free) — these are the **edges**. This is "4-connectivity."

```
# The 4 possible moves from any cell (x, y)
directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
#              right    left      down     up
```

3.2 Core Data Structures

Every graph search algorithm needs three things:

- **Open set (frontier):** Nodes we've discovered but haven't explored yet
- **Came-from map:** For each node, which node did we arrive from? (Used to reconstruct the path)
- **g-score map:** The cheapest cost we've found so far to reach each node from the start

3.3 The Algorithm Family Tree

Algorithm	How it picks the next node	Optimal?	Efficient?
BFS	First-in-first-out queue	Yes (uniform cost)	No — explores in all directions equally
Dijkstra	Lowest actual cost g(n)	Yes	Medium — ignores goal location
Greedy Best-First	Lowest estimated remaining cost h(n)	**No**	Fast but can find bad paths
A*	**Lowest g(n) + h(n)**	**Yes**	**Yes — best of both worlds**

A* is the sweet spot: it's both optimal AND efficient because it balances what it knows (actual cost so far) with what it estimates (remaining cost to goal).

Part 4 — The A* Algorithm: Deep Dive

4.1 The Core Formula

A* evaluates every node with:

```
f(n) = g(n) + h(n)
```

- **g(n):** Actual cost of the cheapest path found from **start → n**
- **h(n):** Heuristic *estimate* of the cost from **n → goal**
- **f(n):** Estimated total cost of the best path **through n**

The node with the **lowest f-score** gets explored next — that's the job of the priority queue.

4.2 The Heuristic: Manhattan Distance

For 4-directional movement on a grid:

```
def heuristic(cell, goal):
    return abs(cell[0] - goal[0]) + abs(cell[1] - goal[1])
```

Why Manhattan? With only horizontal/vertical moves, you *cannot* reach the goal in fewer steps than |Δx| + |Δy|. So this heuristic **never overestimates** — making it *admissible*.

Why admissibility matters: If h(n) ever overestimates, A* might skip the optimal path because it looks "too expensive." An admissible heuristic guarantees A* always finds the shortest path.

Other heuristics for reference:

- **Euclidean:** `sqrt(dx² + dy²)` — use with diagonal movement
- **Chebyshev:** `max(|dx|, |dy|)` — use with 8-directional, uniform-cost diagonals
- **h(n) = 0:** Makes A* behave identically to Dijkstra's

4.3 Pseudocode

```
INITIALIZE:
    open_set ← priority queue with (f=0, start)
    came_from ← empty dictionary
    g_score[all cells] ← infinity
    g_score[start] ← 0

LOOP:
    while open_set is not empty:
        current ← pop node with lowest f-score

        if current == goal → RETURN reconstruct_path(came_from, current)

        for each neighbor of current:
```

```
            if neighbor is in-bounds AND free (grid value = 1):
                tentative_g = g_score[current] + 1

                if tentative_g < g_score[neighbor]:
                    came_from[neighbor] = current
                    g_score[neighbor] = tentative_g
                    f = tentative_g + heuristic(neighbor, goal)
                    add (f, neighbor) to open_set

    RETURN None  (no path exists)
```

4.4 Trace Through a Small Example

```
S . . . .       S = Start (0,0)
. . X . .       G = Goal  (4,4)
. . X . .       X = Obstacle
. . . . .
. . . . G
```

Step 1: Start at S(0,0), g=0. Discover neighbors (1,0) and (0,1), both with g=1.
Step 2: f(1,0) = 1 + |1-4| + |0-4| = 1+7 = 8. f(0,1) = same. Pick either.
Step 3+: Keep expanding the lowest-f node. The obstacle at column 2 forces the path around it. Nodes far from the goal get high f-scores and stay in the queue untouched.

Key insight: The heuristic acts as a "compass" — nodes pointing away from the goal have higher f-scores and get deprioritized.

4.5 The Complete Code

```
from queue import PriorityQueue
import numpy as np

def a_star_search(occupancy_grid, start, goal):

    def heuristic(cell, goal):
        return abs(cell[0] - goal[0]) + abs(cell[1] - goal[1])

    open_set = PriorityQueue()
    open_set.put((0, start))                    # (f-score, node)

    came_from = {}

    # Initialize all g-scores to infinity
    g_score = {cell: float('inf') for cell in
np.ndindex(occupancy_grid.shape)}
    g_score[start] = 0

    while not open_set.empty():
        _, current = open_set.get()             # Pop lowest f-score

        if current == goal:                     # Found the goal!
            return reconstruct_path(came_from, current)
```

```
        # Try all 4 directions
        for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
            x, y = current[0] + dx, current[1] + dy

            # Bounds check AND obstacle check
            if (0 <= x < occupancy_grid.shape[1] and   # x < num_columns
                0 <= y < occupancy_grid.shape[0] and   # y < num_rows
                occupancy_grid[y, x] == 1):             # free cell

                tentative_g = g_score[current] + 1

                if tentative_g < g_score[(x, y)]:      # Found a better path?
                    came_from[(x, y)] = current
                    g_score[(x, y)] = tentative_g
                    f = tentative_g + heuristic((x, y), goal)
                    open_set.put((f, (x, y)))

    return None  # Exhausted all options, no path exists
```

4.6 □ Coordinate Convention — The #1 Bug Source

This is where most students trip up:

- **Cells are stored as (x, y)** → x = column, y = row
- **NumPy indexes as array[row, col]** → `array[y, x]`

So when checking if a cell is free: `occupancy_grid[y, x]` ← Note y first!
And for bounds: x must be < `shape[1]` (columns), y must be < `shape[0]` (rows).

If your path goes through walls, this is almost certainly the bug.

Part 5 — Path Reconstruction

5.1 The came_from Dictionary

Every time A* finds a better path to a neighbor, it records the parent:

```
came_from[(x, y)] = current
```

This creates a chain of links: goal → parent → grandparent → ... → start.

5.2 Tracing the Chain

```
def reconstruct_path(came_from, current):
    path = [current]                    # Start at the goal
    while current in came_from:         # Follow the chain backwards
        current = came_from[current]
        path.insert(0, current)         # Prepend each parent
```

```
    return path                          # Result: [start, ..., goal]
```

Performance note: `path.insert(0, x)` is O(n) per call, making reconstruction O(n²). For large paths, this is faster:

```
def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)             # O(1) amortized
    path.reverse()                       # One O(n) pass at the end
    return path
```

Part 6 — Visualization with Matplotlib

6.1 Display the Grid

```
import matplotlib.pyplot as plt

plt.imshow(occupancy_grid, cmap='gray', origin='lower')
plt.colorbar()
plt.grid(True)
plt.show()
```

`origin='lower'` is crucial — without it, row 0 appears at the top (image convention), not the bottom (math convention). We want y to increase upward.

6.2 Overlay Start, Goal, and Path

```
# Green = Start, Red = Goal
plt.plot(start[0], start[1], 'go', label='Start')
plt.plot(goal[0], goal[1], 'ro', label='Goal')

# Blue = Path
plt.plot([c[0] for c in path_found],    # x-coordinates
         [c[1] for c in path_found],    # y-coordinates
         marker='o', markersize=4, color='blue', label='Path')

plt.legend()
plt.title("A* Path on Occupancy Grid")
plt.show()
```

Since our path stores (x, y) tuples and matplotlib's plot takes (x_list, y_list), the coordinates naturally align.

Part 7 — Performance Improvements

7.1 Problem: Pre-allocating g_score

```
# Current: creates 160,000 entries for a 400×400 grid before search begins
g_score = {cell: float('inf') for cell in np.ndindex(occupancy_grid.shape)}
```

Fix: Use defaultdict

```
from collections import defaultdict
g_score = defaultdict(lambda: float('inf'))
g_score[start] = 0
# Entries created only when accessed — huge memory/time savings
```

7.2 Problem: No Closed Set

Without tracking visited nodes, the same node can be popped from the queue and expanded multiple times.

Fix: Add a closed set

```
closed_set = set()

while not open_set.empty():
    _, current = open_set.get()

    if current in closed_set:
        continue                  # Already fully explored
    closed_set.add(current)

    if current == goal:
        return reconstruct_path(came_from, current)
    # ... rest of loop
```

7.3 Extension: Diagonal Movement

Add 4 diagonal directions and use proper step costs:

```
import math
directions = [
    (1,0), (-1,0), (0,1), (0,-1),          # cardinal: cost 1.0
    (1,1), (1,-1), (-1,1), (-1,-1)          # diagonal: cost √2 ≈ 1.414
]

for dx, dy in directions:
    step_cost = math.sqrt(dx**2 + dy**2)
    tentative_g = g_score[current] + step_cost
```

When allowing diagonals, switch the heuristic from Manhattan to Euclidean or Chebyshev distance for admissibility.

Part 8 — Exercises: Build It Yourself

Exercise 1 (Beginner): Manual Grid

Create a small grid by hand, add walls, run your A* implementation:

```
grid = np.ones((10, 10), dtype=int)
grid[3, 2:8] = 0    # horizontal wall
grid[6, 2:8] = 0    # another wall
start, goal = (1, 1), (8, 8)
```

Exercise 2 (Intermediate): BFS vs A* Comparison

Implement both algorithms. Count nodes expanded by each. Visualize explored regions in light blue. You should see A* exploring far fewer nodes.

Exercise 3 (Intermediate): Diagonal A*

Add 8-directional movement with √2 cost diagonals. Switch to Euclidean heuristic. Compare path length and shape vs 4-directional.

Exercise 4 (Advanced): Dynamic Re-planning

After finding a path, programmatically add an obstacle that blocks it. Rerun A*. Display old path (dashed red) and new path (solid blue) on the same plot.

Exercise 5 (Advanced): Obstacle Inflation

Real robots have physical size. Before pathfinding, expand all obstacles by a safety margin:

```
import scipy.ndimage
obstacle_mask = (occupancy_grid == 0).astype(np.uint8)
inflated = scipy.ndimage.binary_dilation(obstacle_mask, iterations=5)
safe_grid = np.where(inflated, 0, 1)
```

Exercise 6: Create Your Own Test Image

```
img = np.ones((400, 400), dtype=np.uint8) * 255
cv2.rectangle(img, (100,50), (120,300), 0, -1)
cv2.rectangle(img, (200,100), (220,350), 0, -1)
cv2.rectangle(img, (300,50), (320,250), 0, -1)
cv2.imwrite("my_grid.png", img)
```

Part 9 — Summary: Key Takeaways

1. **Occupancy grids** turn images into binary matrices a computer can reason about (1=free, 0=blocked).
2. **Thresholding** is the bridge — it decides the cutoff between free and occupied.
3. **A* = g(n) + h(n)** — combines known cost with estimated remaining cost.
4. **Manhattan distance** is the correct heuristic for 4-directional grids because it's admissible.
5. **Priority queues** ensure we always expand the most promising node next.
6. **Path reconstruction** follows the came_from chain backwards from goal to start.
7. **Coordinate conventions** (x,y vs row,col) are the most common source of bugs.

The concept chain students need to master:

```
Image → Threshold → Occupancy Grid → Implicit Graph →
  Search (BFS → Dijkstra → A*) →
    Heuristics → Priority Queue → Path Reconstruction → Visualization
```